A Verification and Playing-Evidence Inventory for the Unchessed UCI Chess Engine

Manus AI
Verification and Research Facilities Unchessed-UCI-Engine Branch: `manus/research-facilities`

***Abstract*—This paper presents a complete evidence inventory for the Unchessed Universal Chess Interface (UCI) engine. The inventory separates deterministic automated tests, source and artifact audits, real release-binary experiments, and actual playing evidence. The current evidence includes 123 passing Rust tests, 385 passing Python tests with 22 skips, a 21-file Rust bracket audit, a deterministic CPU calibration, NNUE-loaded searches, UCI lifecycle probes, fixed-position experiments, and negative capability tests for tablebases, pondering, Chess960, quantized inference, and other features. The central result is a boundary result: the repository is extensively checked as software, but the current Tier 2 calibration is explicitly a toy non-chess plumbing experiment. No human-versus-engine game was played, and no new engine-versus-engine chess match or Elo experiment was run in this task. A prior interrupted Persona-related match reached approximately 5,537 games without an SPRT decision, and historical small SPSA experiments remain underpowered. Consequently, fixed positions, NPS, source inspection, toy learning, and regression tests must not be reported as playing-strength evidence.**

***Index Terms*—chess engine, UCI, software verification, NNUE, regression testing, self-play, SPRT, reproducibility**

I. Introduction

A chess engine requires more than a passing unit-test suite. Board legality, search termination, evaluation, protocol behavior, model loading, timing, concurrency, and playing strength are related but distinct properties. A test that establishes one property must not be silently promoted into evidence for another. This paper consolidates the complete test and playing inventory for the Unchessed-UCI-Engine repository, following the evidence boundaries in the project research records [1]–[3].

The paper addresses four questions. First, which automated suites have actually run, and what did they establish? Second, which experiments exercised the real release binary against real positions or protocol sequences? Third, which data and source audits were completed? Fourth, did a human or another chess engine actually play games against Unchessed in the current work? The final question is deliberately explicit because playing evidence is often conflated with smoke testing.

The paper's main conclusion is conservative. The software and research facilities have substantial automated coverage, and the upgraded Rust workspace passes its current regression suite. However, the most recent Tier 2 CPU calibration is a toy transition system that does not load the NNUE, does not implement chess self-play, and makes no Elo claim. The evidence therefore supports implementation correctness and reproducibility, not a strength increase.

II. Evidence Taxonomy

A. Evidence levels

We use five evidence levels. *Source evidence* means that code, configuration, or an artifact was inspected. *Automated evidence* means that a repeatable test or checker ran to completion. *Runtime evidence* means that a release binary processed a real position or UCI command sequence. *Playing evidence* means that two players or engines completed a declared set of chess games under fixed conditions. *Promotion evidence* means a provenance-complete paired-game experiment, normally analyzed with a predeclared sequential probability ratio test (SPRT), that is sufficient for a game-facing candidate decision. These levels are not interchangeable.

For example, a fixed FEN search can establish that a binary accepts the FEN and returns a legal-looking result. It cannot establish that an evaluation term increases Elo. Similarly, a node count can establish a measured runtime property for one configuration, but not a general speedup or playing-strength change. The project therefore treats a real paired-game SPRT as the required boundary for a normal-search default or model promotion [4], [5].

B. Reproducibility context

All current work was performed on the non-main branch `manus/research-facilities`. The repository is clean and synchronized with its remote branch at the time of writing. The Rust toolchain used for the final regression run is `rustc 1.98.0` and `cargo 1.98.0`. Earlier reports sometimes record a Cargo 1.75 lockfile incompatibility because individual investigations used an older executable path; the final workspace result in this paper used the upgraded toolchain.

III. Automated Test Suites

A. Rust workspace regression

The release workspace suite was executed as follows:

```
. "$HOME/.cargo/env"
cargo test --workspace --release
```

The result was 123 passed, zero failed, and six ignored tests. The suite exercised the core crate, binary targets, and documentation-test targets. No failure was hidden by a logging pipeline or an unconditional success operator.

Table I summarizes the principal Rust coverage areas. The list describes tested behavior, not a claim that every possible position or race is exhaustively covered.

TABLE I
PRINCIPAL RUST REGRESSION AREAS

Area	Examples of verified behavior
Board and rules	FEN, move making, legal generation, SAN, perft, stalemate
Search	mate in one/two, forced move, repetition, time budget, node limits, MultiPV
Tactics	SEE exchanges, pins, x-rays, quiescence, checking and evasion safeguards
Evaluation	static evaluation, NNUE non-degeneracy, incremental state coverage
TT and UCI	hashfull, prefetch, option parsing, lifecycle, threads, root hints
Safety paths	Unarchitectured package parsing, corruption and truncation rejection

More specifically, search tests include legal fallback behavior under node limits, forced mate handling, root-hint safety, distinct MultiPV moves, and timing-budget behavior. SEE tests include pinned recapturers and three-ply x-ray exchanges. UCI tests cover option parsing, empty values, thread defaults, telemetry defaults, persona defaults, game resets, and Unarchitectured hint behavior.

B. Python tooling suite

The complete tooling suite was executed using:

```
python3 -m pytest tools/ -q
```

It produced 385 passed tests, 22 skipped tests, and 347 passed subtests. The repository contains 42 Python test modules spanning training controls, data blocks, evaluation diagnostics, policy priors, persona telemetry, SPRT calculations, search consistency, NNUE relabeling, Unarchitectured runtime and safety, and the CPU calibration harness.

The skipped tests are not counted as passes. Several are intentionally conditional on unavailable hardware, cloud services, data, or external binaries. This distinction is important because a skipped A100 or cloud self-play test does not establish that the corresponding facility works locally.

C. Focused CPU calibration suite

The focused Tier 2.2 suite was executed with:

```
pytest tools/test_rl_calibration.py -q
```

All four tests passed. They verify deterministic replay generation, legal-action mask adherence, held-out value-loss improvement, machine-readable content-addressed output, and fail-closed rejection of an invalid game bound.

The calibration run used 100 games, seed 17, 25 updates, and learning rate 0.05. It completed 100 games and generated 451 replay records. Legal-mask violations were zero. Held-out loss fell from 0.22666666666666666 to 4.4702117962524e-32. The canonical replay SHA-256 begins `c6dfbf10d7896c95`; the complete digest is recorded in the repository report [3].

This result is intentionally limited. The environment is an acyclic toy transition system. It is not chess, does not load `unchessed-nnue.bin`, does not implement Monte Carlo Tree Search or PUCT, and does not train a policy/value architecture. Thus, the loss decrease establishes only that a small deterministic replay/value-learning plumbing path can be tested.

D. Structural checker

The Rust bracket checker was run with the `--all` option. It checked 21 Rust files and reported all 21 as balanced. This is useful as a cheap structural sanity check, but it is weaker than compilation and does not establish semantics.

IV. REAL-WORLD RUNTIME TESTING

A. NNUE-loaded searches

The release adapter was run against real positions with the shipped NNUE file. The UCI sequence included `setoption name EvalFile`, `isready`, start-position searches, and a middlegame FEN. The binary reported successful NNUE loading, returned search information, and produced best moves. One fixed one-thread timed run reached depth 11 with 657,832 nodes, 885,372 search NPS, and 743 ms.

These measurements establish that the tested binary can load the named model and search the stated positions. They do not isolate forward-pass speed, prove AVX2 branch execution, establish model accuracy, or imply Elo.

B. Protocol and lifecycle probes

Real UCI probes exercised `uci`, `isready`, `position`, `go depth`, `go nodes`, `go movetime`, `stop`, `quit`, `setoption`, `Clear Hash`, and `ucinewgame`. The observed responses included `uciok`, `readyok`, search information, and best moves. Hash resize and reset lifecycle commands completed in the tested binary.

The probes also tested unsupported or incomplete features. `go ponder` and `ponderhit` produced ordinary search behavior rather than a ponder state transition. Shredder-style Chess960 rights `AHah` were rejected, and no `UCI_Chess960` option was advertised. A Python Syzygy probe was attempted after installing `python-chess`; it failed with `FileNotFoundError` because no tablebase directory or files were available. These are valuable negative runtime results, not failures of hypothetical implementations.

C. Fixed-position experiments

Four pawn-structure FENs were searched and reported scores including 142, 250, 136, and 252 cp. A Contempt matrix at values 0, 25, and 100 with adaptive mode on and off showed that adaptive mode could alter PV and node behavior. A fixed-FEN Lazy SMP experiment at 1, 2, 4, and 6 threads reported 321,477, 236,853, 198,458, and 241,130 nodes, respectively, with times of 178, 137, 120, and 132 ms.

These are real executions, but the fixtures are not a substitute for a game suite. Pawn scores are confounded by material and other evaluation terms. Contempt behavior is not a draw-rate

study. Thread counts and NPS on one position do not establish the best SMP schedule. Fixed-position results are diagnostic evidence only.

D. Runtime safety and capability checks

The AVX2 audit found runtime feature gates around the inspected floating SIMD paths and no confirmed safety defect. Local feature detection reported AVX2, FMA, and SSE4.1. However, one later source-build attempt was blocked by the lockfile/toolchain path, and a separate available-binary smoke lacked the NNUE file. Therefore, a fully current-source, NNUE-loaded, AVX2 empirical test remains incomplete.

A real `Hash=1` lifecycle smoke completed readiness, search, `Clear Hash`, and `ucinewgame` commands. A fixed-bench investigation ran real node-limited searches, but current output did not provide sufficiently stable aggregate final-node reporting for a proper NPS regression suite. This motivates a future developer bench rather than a claim that one already exists.

V. Source, Data, and Artifact Audits

The research pass scanned the engine for tablebase hooks, pondering, Chess960, quantized NNUE inference, a standard bench command, TT memory auto-sizing, and AVX2 unsafe blocks. It also inspected UCI option flow, search pruning, NNUE architecture and record formats, training controls, and provenance documentation.

The audits corrected two stale assumptions. First, check extensions already exist: the search has an in-check depth floor and an explicit extension for checking moves. Second, qsearch already skips strictly losing captures according to SEE at non-check nodes. These corrections illustrate why source inspection must precede feature planning.

The NNUE label-noise record was also corrected using a real paired measurement of 2,900 positions: 17.41–21.99 cp mean absolute error and 0.929–0.983 Pearson correlation. The earlier modeled 50–56 cp floor is retired for the tested setup. Separately, the later piece-count resampling/reweighting thread was closed at the tested scale after mixed hard-resampling results and a negative clipped soft-weighting result. The supplied experiment figures were not recomputed in this checkout and are labeled accordingly.

VI. Human and Engine Playing Evidence

A. Human play

No human-versus-engine or human-versus-human chess game was played in the current task. The assistant did not operate a chessboard as a human player, select moves interactively, or complete a human game against the engine. Position analysis and interpretation of UCI output are not human playing evidence.

B. Engine-versus-engine play

The original Tier 2.2 CPU calibration rerun used toy episodes rather than chess positions. A subsequent bounded Cutechess gate launched real chess games. Four Unchessed-versus-Unchessed adapter games completed successfully, producing two wins for each binary when colors were alternated plus one draw by insufficient mating material; the aggregate was 2–1–1 for the first named adapter. A second four-game gate paired Unchessed with packaged Stockfish 16. All four games completed: Unchessed scored 0–4–0, with two losses as White and two as Black. Both gates used one thread, 16 MiB hash, disabled Unchessed book, the shipped NNUE for Unchessed, and 1+0.1 time control. PGNs and logs are retained in the repository. These are bounded runtime and game-completion observations, not a statistically powered strength claim or SPRT decision.

TABLE II
Playing-evidence status

Evidence type	Status in this work
Human versus engine	Not run
Human versus human	Not run
New engine versus engine chess match	Run: two bounded 4-game gates
Chess self-play for RL	Not run
Toy CPU calibration episodes	Run; explicitly not chess
Historical Persona match	Approximately 5,537 games; no SPRT decision
Historical small SPSA work	Recorded; underpowered
Promotion-quality paired SPRT	Not run in this task

Earlier repository work contains historical playing evidence that must remain separate from these bounded gates. A prior combined Persona-related experiment stopped at approximately 5,537 games without reaching an SPRT decision. Historical search/tuning work also contains small SPSA iterations, including a 12-game experiment. These records, like the new four-game Stockfish gate, are not statistically powered strength evidence and cannot justify a default change.

C. What would constitute playing evidence

A meaningful engine comparison would freeze the exact binaries, NNUE files, UCI options, opening set, hardware, thread count, hash size, time control, adjudication rules, and randomization policy. It would retain raw PGNs, engine logs, manifests, and result recomputation inputs. The candidate would then be compared against the incumbent in paired games and analyzed with a declared SPRT or equivalent statistical plan. A continuous controller can improve operations, but it does not replace provenance or statistical validity [5], [6].

VII. Limitations and Reproducibility

The inventory is comprehensive for the recorded repository work, but it is not a proof of chess correctness over the entire game tree. The Rust suite contains ignored tests, the Python suite contains skips, and some external-data and hardware paths are unavailable locally. A release-binary smoke can differ from a current-source build if assets or toolchain paths differ. In particular, the current workspace pass should not erase the negative toolchain records in earlier feature reports; those records accurately describe the commands and environments used at that time.

The NNUE throughput question remains unmeasured in the batched form required for RL. The toy calibration does not remove the independent architecture and throughput blockers. Likewise, fixed-position search and score experiments are useful for debugging but do not provide a distribution of game outcomes. No claim in this paper should be interpreted as an Elo estimate.

VIII. Conclusion

Unchessed has broad automated verification and a growing body of real runtime evidence. The final upgraded-toolchain Rust workspace run passed 123 tests, the Python tooling suite passed 385 tests with 22 skips, and all 21 Rust files passed the structural balance check. Real release-binary probes verified model loading, UCI lifecycle behavior, hash handling, fixed-position searches, and several negative capabilities.

The decisive boundary is playing evidence. No human game and no new engine-versus-engine chess match occurred in the current Tier 2.2 work. The only fresh calibration episodes were toy episodes designed to test replay and value-learning plumbing. A prior interrupted match and historical small tuning experiments are recorded but do not establish a strength result. Future game-facing changes therefore require a separately declared, provenance-complete paired-game campaign and statistical decision rule.

References

[1] Unchessed AI, "Tier 1 master synthesis: evidence, diagnostics, and promotion boundaries," project report, 2026.

[2] Unchessed AI, "Tier 1 update 3 synthesis: completed evidence, corrections, and promotion boundaries," project report, 2026.

[3] Unchessed AI, "Tier 2.2 bounded CPU calibration prototype," project report, 2026.

[4] M. B. Trivedi, "Sequential probability ratio testing for chess-engine comparisons," project methodology record, 2026.

[5] Stockfish Developers, "Fishtest," official project documentation. [Online]. Available: https://official-stockfish.github.io/docs/fishtest/

[6] Stockfish Developers, "Fastchess," official project repository. [Online]. Available: https://github.com/Disservin/fastchess

[7] S. Meyer-Kahlen, "Universal Chess Interface protocol," protocol specification. [Online]. Available: https://www.shredderchess.com/chess-info/features/uci-universal-chess-interface.html

[8] Ronald de Man, "Syzygy endgame tablebases," technical documentation. [Online]. Available: https://syzygy-tables.info/

[9] Stockfish Developers, "NNUE," official technical documentation. [Online]. Available: https://official-stockfish.github.io/docs/nnue-pytorch-wiki/docs/nnue.html